LUMA Project - Architecture & Step-by-Step Code Guide

Personal Learning Log & Chronological Building Blueprint | Updated Daily

---

1. Tech Stack & Architecture Overview

LUMA is a modern fullstack web application and high-performance HTML5 canvas game built with modern TypeScript web technologies.

Layer	Technology	Purpose
Frontend UI	React 19 + TypeScript	Modular UI components & strict type safety
Game Engine	HTML5 Canvas 2D API	120 FPS game loop, particles, render pipeline
Framework	TanStack Start	Fullstack SSR framework & file-based routing
Styling	Tailwind CSS v4 + Radix UI	Utility styling & accessible UI primitives
Database & Auth	Supabase (PostgreSQL)	Managed database, Auth & Row Level Security
Build Engine	Vite v8 + Nitro Server	Fast dev server and cloud production bundle

2. Chronological Code Construction Sequence (From Scratch)

When building LUMA from scratch after setting up dependencies, files MUST be written in this exact dependency order:

Step	File to Write First	Architectural Rationale
Step 1	src/game/types.ts	Define data contracts (Player, Enemy, Bullet, GameData). All other files import these types.
Step 2	src/game/engine.ts	Build game state factory & 120 FPS physics update logic (movement, collisions, powerup timers).
Step 3	src/game/audio.ts	Synthesize retro sound FX (lasers, explosions, powerup chimes) using browser Web Audio API.
Step 4	src/game/render.ts	Create Canvas 2D graphics renderer (starfield, player ship, particles, lasers, HUD overlay).
Step 5	src/components/NovaBlaster.tsx	React container component mounting , keyboard/touch input, & requestAnimationFrame loop.
Step 6	src/router.tsx & src/routes/index.tsx	TanStack router configuration & root homepage route mounting NovaBlaster UI with SEO meta tags.
Step 7	src/start.ts & src/server.ts	Fullstack SSR hydration & backend Supabase database integration for leaderboard high scores.

3. Configuration Files & System Folders

File / Folder	Type	Description & Function

.lovable/	Lovable AI Folder	Stores AI project metadata, revision IDs, and prompt execution plans.
.output/	Build Output	Generated by Nitro / Vite when building production server bundle.
.tanstack/	Framework Cache	Auto-generated manifest & caching directory for TanStack Router.
components.json	Shadcn UI Config	Defines component paths, CSS variables, and utility aliases (@/lib/utils).
supabase/	Database & Backend	Contains SQL migrations, database table schemas, and RLS security rules.